

VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering

CAPSTONE PROJECT REPORT

Library Management System

A Full-Stack Web Application using ASP.NET Core 8 MVC and SQL Server

SOURCE CODE REPOSITORY

<https://github.com/Kavyansh180/Library-Management-System>

SUBMITTED BY

Student Name	Kavyansh Vats
Application Number	IN26013045
Registration Number	23BCE10301
Programme	B.Tech — Computer Science and Engineering

FRAMEWORK	DATABASE	ACADEMIC YEAR
ASP.NET Core 8	SQL Server Express	2025 – 2026



Abstract

This report presents the design, development, and deployment of a Library Management System (LMS) built as a full-stack web application using the ASP.NET Core 8 MVC framework and Microsoft SQL Server Express. The system provides an integrated digital platform for managing library resources including books, students, librarians, borrowing records, and publications such as newspapers and magazines.

The application follows the Model-View-Controller (MVC) architectural pattern, leveraging Entity Framework Core 8 as the Object-Relational Mapper for structured data access, supplemented by parameterised raw SQL queries for performance-critical operations. ASP.NET Core Identity is integrated for authentication infrastructure. The user interface is built with Bootstrap 5.3 and Razor view templates, delivering a responsive and browser-agnostic experience.

The project was developed, tested, and version-controlled using Git with the complete source code hosted on GitHub. All major modules — book cataloguing, borrowing and return workflow, student and librarian management, publications tracking, and a real-time statistical dashboard — were implemented and verified against a live SQL Server Express instance.

Keywords: ASP.NET Core 8, MVC, Entity Framework Core, SQL Server, Library Management, C#, Razor Views, Bootstrap 5

Table of Contents

1.	Introduction and Objectives	3
2.	Technology Stack	3
3.	System Architecture	4
4.	Project Structure	4
5.	Database Design	5
6.	Modules and Feature Description	6
7.	Controllers and Business Logic	7
8.	Views and User Interface	8
9.	Setup and Installation Guide	9

10.	Testing and Verification	9
11.	Conclusion and Future Scope	10

Capstone Project Report

Library Management System

Page 2

SECTION 1

Introduction and Objectives

Traditional library management relies heavily on manual record-keeping processes that are error-prone, time-consuming, and difficult to scale. This project addresses those challenges by delivering a fully digital, web-based Library Management System that automates the core administrative functions of a library.

The system is accessible through any standard web browser, requires no client-side installation, and is backed by a robust relational database. It centralises the management of books, student and librarian records, borrowing and return workflows, and publication archives within a single, cohesive platform.

1.1 Objectives

- Design and implement a multi-module web application using the ASP.NET Core 8 MVC pattern.
- Provide complete CRUD (Create, Read, Update, Delete) functionality for books, students, librarians, and publications.
- Implement a borrowing and return workflow with real-time availability tracking.
- Deliver a real-time administrative dashboard aggregating statistics from all modules.
- Integrate ASP.NET Core Identity for secure authentication infrastructure.
- Manage source code with Git and publish to a public GitHub repository.
- Ensure the system is maintainable, extensible, and follows software engineering best practices.

1.2 Scope

The scope of this capstone project encompasses the full software development lifecycle including requirements analysis, system design, implementation, database migration, testing, and version control. The system was developed and tested on a Windows environment with .NET 8.0 SDK and SQL Server Express 2017.

SECTION 2

Technology Stack

Category	Technology	Version	Role
Backend Framework	ASP.NET Core MVC	8.0	Web application and routing framework
Programming Language	C#	.NET 8.0	Application business logic
ORM	Entity Framework Core	8.0.0	Data access for Books, BorrowRecords, Publications
Identity	ASP.NET Core Identity	8.0.0	User authentication and role infrastructure
Raw Data Access	Microsoft.Data.SqlClient	5.1.1	Parameterised SQL for Students and Librarians
Database	SQL Server Express	2017 / 2022	Primary relational data store
Scaffolding	Microsoft.VisualStudio.Web.CodeGeneration.Design	8.0.0	Controller and view scaffolding
Frontend	Bootstrap	5.3.0 (CDN)	Responsive UI layout and components
Frontend	jQuery	3.6.0 (CDN)	DOM manipulation and AJAX support
Templating	Razor (.cshtml)	Built-in	Server-side HTML rendering
Version Control	Git / GitHub	—	Source code management and collaboration
Development Runtime	Kestrel (dotnet run)	Built-in	Cross-platform HTTP server

SECTION 3

System Architecture

The application adheres strictly to the Model-View-Controller (MVC) architectural pattern enforced by the ASP.NET Core framework. This pattern enforces a clean separation of concerns: the Model layer encapsulates domain entities and data access logic; the View layer contains Razor templates responsible solely for HTML rendering; and the Controller layer handles HTTP request routing, orchestrates data retrieval, and selects the appropriate view response.

3.1 Request-Response Flow

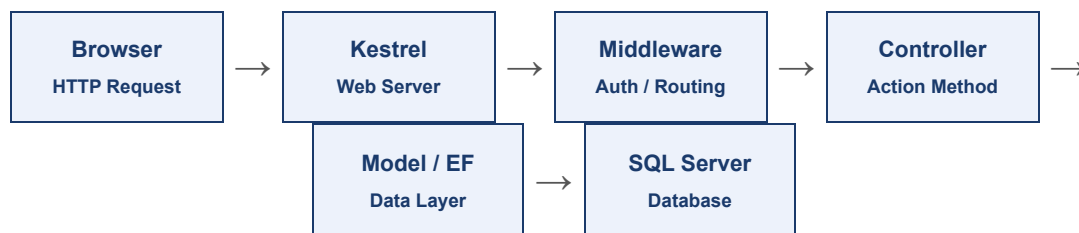


Figure 3.1: Application request-response flow

3.2 Data Access Strategy

The project employs a dual data access strategy. Entity Framework Core is used for the `Books13`, `BorrowRecords13`, and `Publications` tables, where the ORM's change tracking and relationship navigation provide productivity benefits. For the `Students` and `Librarians` tables, parameterised raw SQL via `Microsoft.Data.SqlClient` is used directly, demonstrating proficiency with both abstracted and explicit database interaction patterns.

3.3 Dependency Injection

ASP.NET Core's built-in Dependency Injection container is used to register and resolve services. Both `LibraryContext` and `ApplicationDbContext` are registered as scoped services in `Program.cs`, along with the Identity services. Configuration values (connection strings) are injected into controllers via `IConfiguration`.

SECTION 4

Project Structure

```
Library-Management-System/ |-- LMSystem/ ASP.NET Core 8 MVC Project | |--  
Controllers/ 8 controller classes | | |-- BooksController.cs | | |--  
BorrowController.cs | | |-- DashboardController.cs | | |-- HomeController.cs |  
| |-- LibrarianController.cs | | |-- LoginController.cs | | |--  
PublicationsController.cs | | |-- StudentController.cs | |-- Data/ EF Core  
DbContext definitions | | |-- ApplicationDbContext.cs ASP.NET Identity context  
| | |-- LibraryContext.cs Library domain context | |-- Migrations/ EF Core  
migration files | | |-- Identity/ IdentityCoreDB migrations | | |-- Library/  
LMS database migrations | |-- Models/ 7 domain model classes | |-- ViewModels/  
5 view-specific model classes | |-- Views/ 17 Razor view templates (.cshtml) |  
|-- wwwroot/ Static assets (CSS, JS, images) | |-- Program.cs Application  
bootstrap and DI setup | |-- LMSystem.csproj Project file and NuGet references  
| |-- appsettings.json Connection strings and logging config |-- .gitignore `--  
- Library_Management_System_Report.pdf
```

SECTION 5

Database Design

The system uses two SQL Server Express databases managed through distinct strategies. The **LMS** database holds all library domain data; the **IdentityCoreDB** database holds ASP.NET Core Identity tables generated automatically via EF Core migrations.

5.1 LMS Database — Table Definitions

Table: Books13

Column	Data Type	Constraints	Description
BookId	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for each book
Title	NVARCHAR(MAX)	NULL	Title of the book
Author	NVARCHAR(MAX)	NULL	Author name
ISBN	NVARCHAR(MAX)	NULL	International Standard Book Number
PublishedDate	DATETIME2	NOT NULL	Date of publication
IsAvailable	BIT	NOT NULL, DEFAULT 1	Availability flag; 0 when borrowed

Table: BorrowRecords13

Column	Data Type	Constraints	Description
BorrowRecordId	INT	PRIMARY KEY, IDENTITY(1,1)	Unique borrow record identifier
BookId	INT	FOREIGN KEY -> Books13(BookId)	Book being borrowed
BorrowerName	NVARCHAR(MAX)	NULL	Full name of the borrower

Column	Data Type	Constraints	Description
BorrowerEmail	NVARCHAR(MAX)	NULL	Email address of the borrower
Phone	NVARCHAR(MAX)	NULL	Contact phone number
BorrowDate	DATETIME2	NOT NULL, DEFAULT UTC	Date and time of borrowing
ReturnDate	DATETIME2	NULL	Date returned; NULL until book is returned

Table: Publications

Column	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY(1,1)	Unique publication identifier
Title	NVARCHAR(MAX)	NULL	Publication title
Publisher	NVARCHAR(MAX)	NULL	Publisher name
PublishedDate	DATETIME2	NOT NULL	Date of publication
Type	INT	NOT NULL	Enum: 0 = Newspaper, 1 = Magazine
IsAvailable	BIT	NOT NULL	Availability status

Table: Students

Column	Data Type	Constraints	Description
StudentId	INT	PRIMARY KEY, IDENTITY(1,1)	Unique student identifier
Name	NVARCHAR(100)	NOT NULL	Full name of the student
Age	INT	NOT NULL	Age of the student
Phone	NVARCHAR(20)	NOT NULL	Contact phone number

Table: Librarians

Column	Data Type	Constraints	Description
LibrarianId	INT	PRIMARY KEY, IDENTITY(1,1)	Unique librarian identifier
Name	NVARCHAR(100)	NOT NULL	Full name
Age	INT	NOT NULL	Age
Phone	NVARCHAR(20)	NOT NULL	Contact phone number

5.2 IdentityCoreDB — ASP.NET Identity Tables

The identity database is created and managed entirely by EF Core migrations. Key tables include `AspNetUsers` (user accounts with hashed passwords), `AspNetRoles` (role definitions), `AspNetUserRoles` (user-role mapping), `AspNetUserClaims`, `AspNetUserLogins`, and `AspNetUserTokens`.

SECTION 6

Modules and Feature Description

6.1 Book Management Module

Managed by `BooksController` using Entity Framework Core and the `LibraryContext`. Provides full CRUD operations: listing all books with paginated output, searching by title or author, adding new book entries, editing existing records, and deletion with cascade effect on associated borrow records. Each book's `IsAvailable` flag is toggled automatically during borrow and return transactions.

6.2 Borrow and Return Workflow

Managed by `BorrowController` using async EF Core operations. When a student borrows a book, the system verifies the book exists and its `IsAvailable` flag is `true`. A `BorrowRecord` is created, and the book is marked unavailable. On return, `ReturnDate` is set to the current UTC timestamp and the book is re-marked as available. All operations are protected by AntiForgery tokens.

6.3 Student Management Module

Managed by `StudentController` using raw parameterised SQL via `SqlConnection`. Supports server-side search filtering by name and paginated listing (5 records per page) using SQL `OFFSET / FETCH NEXT` syntax to prevent SQL injection.

6.4 Librarian Management Module

Managed by `LibrarianController` with raw SQL and `IConfiguration`-injected connection strings. Full CRUD is supported: listing with pagination, name-based search, create, edit, and delete via direct SQL statements with parameterised values.

6.5 Publications Module

Managed by `PublicationsController` using EF Core with the `Publication` entity. Publications are categorised by a `PublicationType` enum (0 = Newspaper, 1 = Magazine). The index view accepts a `type` query parameter and filters accordingly. Pagination is implemented server-side.

6.6 Dashboard Module

Managed by `DashboardController` using raw SQL scalar queries across all five data tables. Each count query is enclosed in an individual `try-catch` block to ensure partial data still renders if a single table query fails. Connection strings are injected via `IConfiguration`.

6.7 Authentication Module

Managed by `LoginController`. Provides a login form and verifies credentials against an in-memory list (suitable for demonstration purposes). On successful authentication, the user is redirected to the dashboard. The ASP.NET Core Identity middleware pipeline (`UseAuthentication`, `UseAuthorization`) is fully configured in `Program.cs` for future extension.

6.8 Feature Completion Summary

Feature	Controller	Data Access	Status
Book CRUD with pagination	BooksController	EF Core	COMPLETE
Borrow and Return workflow	BorrowController	EF Core (async)	COMPLETE
Student CRUD with search	StudentController	Raw SQL	COMPLETE
Librarian CRUD with search	LibrarianController	Raw SQL	COMPLETE
Publications with type filter	PublicationsController	EF Core	COMPLETE
Real-time dashboard statistics	DashboardController	Raw SQL	COMPLETE
Login and authentication	LoginController	In-memory	COMPLETE
Responsive Bootstrap UI	Shared Layout	—	COMPLETE
EF Core database migrations	—	EF Core CLI	COMPLETE
GitHub version control	—	Git	COMPLETE
Role-based access control	—	Identity	FUTURE SCOPE
Fine management system	—	—	FUTURE SCOPE

SECTION 7

Controllers and Business Logic

7.1 Dependency Injection Configuration (Program.cs)

The application startup registers both DbContexts and the Identity services using the built-in DI container. Connection strings are read from `appsettings.json`:

```
// Register EF Core contexts with SQL Server provider
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("SQLServerIdentityConnection")));

builder.Services.AddDbContext<LibraryContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

// Register ASP.NET Core Identity
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();

// Middleware pipeline
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

7.2 Borrow Workflow — BorrowController

```
public async Task<IActionResult> Create(BorrowViewModel model)
{
    var book = await _context.Books13.FindAsync(model.BookId);
    if (book == null) return View("NotFound");
    if (!book.IsAvailable) return View("NotAvailable");

    var record = new BorrowRecord {
        BookId = book.BookId,
        BorrowerName = model.BorrowerName,
        BorrowerEmail = model.BorrowerEmail,
        Phone = model.Phone,
```



```

        BorrowDate      = DateTime.UtcNow
    };
    book.IsAvailable = false;
    _context.BorrowRecords13.Add(record);
    await _context.SaveChangesAsync();
    return RedirectToAction("Index", "Books");
}

```

7.3 Paginated Search — LibrarianController

```

string dataQuery = @"
    SELECT * FROM Librarians
    WHERE (@SearchTerm IS NULL OR Name LIKE '%' + @SearchTerm + '%')
    ORDER BY LibrarianId
    OFFSET @Offset ROWS FETCH NEXT @PageSize ROWS ONLY";

cmd.Parameters.AddWithValue("@SearchTerm", (object?)searchTerm ?? DBNull.Value);
cmd.Parameters.AddWithValue("@Offset", offset);
cmd.Parameters.AddWithValue("@PageSize", pageSize);

```

7.4 Configuration — appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
      "Data Source=.\SQLEXPRESS;Initial Catalog=LMS;
      Integrated Security=True;Encrypt=False;TrustServerCertificate=True",
    "SQLServerIdentityConnection":
      "Data Source=.\SQLEXPRESS;Initial Catalog=IdentityCoreDB;
      Integrated Security=True;Encrypt=False;TrustServerCertificate=True"
  }
}

```

Security Note: The demonstration login credentials are hardcoded in `LoginController`. In a production deployment, all authentication must be handled exclusively through ASP.NET Core Identity with properly hashed password storage and role-based authorization attributes applied to all sensitive controller actions.

SECTION 8

Views and User Interface

All views are implemented as Razor templates (.cshtml) and share a common master layout defined in `Views/Shared/_Layout.cshtml`. The layout includes a Bootstrap 5.3 navigation bar, a page content container, and a footer. Bootstrap and jQuery are loaded via CDN links within the layout's `<head>` block, ensuring no static file dependencies must be managed locally.

8.1 View Inventory

View File	Controller	Purpose
Home/Index.cshtml	HomeController	Landing page with module navigation cards
Login/Index.cshtml	LoginController	Login form with username and password fields
Dashboard/Index.cshtml	DashboardController	Summary statistics: books, students, librarians, borrowings, publications
Books/Index.cshtml	BooksController	Paginated book table with Borrow, Edit, and Delete actions
Borrow/Create.cshtml	BorrowController	Borrow form collecting borrower name, email, and phone
Borrow/Return.cshtml	BorrowController	Return confirmation page displaying borrow details
Librarian/Index.cshtml	LibrarianController	Searchable paginated list with Edit and Delete actions
Librarian/Create.cshtml	LibrarianController	Add new librarian form with validation
Librarian/Edit.cshtml	LibrarianController	Pre-populated edit form for existing librarian records
Student/Index.cshtml	StudentController	Searchable paginated student list

View File	Controller	Purpose
Publications/Index.cshtml	PublicationsController	Type-filtered publication list (Newspaper / Magazine)
Shared/_Layout.cshtml	All controllers	Master layout: navbar, Bootstrap CDN, jQuery, footer
Shared/NotFound.cshtml	All controllers	Displayed when a requested record is not found
Shared/NotAvailable.cshtml	BorrowController	Displayed when a book is already borrowed
Shared/AlreadyReturned.cshtml	BorrowController	Displayed when a return has already been processed
Shared/Error.cshtml	All controllers	Generic application error page

Table 8.1: Complete view inventory

8.2 Razor Tag Helpers and Validation

All form views use ASP.NET Core Razor tag helpers for clean, maintainable markup: `asp-controller`, `asp-action`, `asp-for`, and `asp-validation-for`. The `@Html.AntiForgeryToken()` helper is included on every POST form to prevent Cross-Site Request Forgery (CSRF) attacks. The `[ValidateAntiForgeryToken]` attribute is applied to all corresponding POST action methods.

8.3 ViewModel Pattern

View-specific data requirements are addressed through dedicated ViewModel classes rather than passing domain models directly to views. For example, `LibrarianIndexViewModel` encapsulates the list of librarians alongside pagination metadata (current page, total pages, page size, search term), keeping the view layer decoupled from data access concerns.

SECTION 9

Setup and Installation Guide

9.1 Prerequisites

.NET 8.0 SDK

Download from `dotnet.microsoft.com`.
Verify installation with `dotnet --version`.

SQL SERVER EXPRESS (ANY EDITION)

SQL Server Express 2017 or later. Ensure the `MSSQL$SQLEXPRESS` service is running.

GIT

Required to clone the repository. Verify with `git --version`.

EF CORE CLI TOOL

Install globally: `dotnet tool install --global dotnet-ef`

9.2 Step-by-Step Installation

```
# Step 1 - Clone the repository
git clone https://github.com/Kavyansh180/Library-Management-System.git
cd Library-Management-System/LMSysstem

# Step 2 - Install EF Core CLI tool (if not already installed)
dotnet tool install --global dotnet-ef

# Step 3 - Apply EF Core migrations (creates LMS and IdentityCoreDB)
dotnet ef database update --context LibraryContext
dotnet ef database update --context ApplicationDbContext

# Step 4 - Create raw SQL tables (Students and Librarians)
sqlcmd -S .\SQLEXPRESS -d LMS -Q "
    IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Students')
    CREATE TABLE Students (StudentId INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(100) NOT NULL, Age INT NOT NULL, Phone NVARCHAR(20) NOT NULL);
    IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Librarians')
    CREATE TABLE Librarians (LibrarianId INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(100) NOT NULL, Age INT NOT NULL, Phone NVARCHAR(20) NOT NULL);"

# Step 5 - Run the application
```

```
dotnet run
# Application is now accessible at: http://localhost:5000
```

Note: The connection strings in `appsettings.json` target `.\SQLEXPRESS` with Windows Integrated Security. If your SQL Server instance name differs, update the `Data Source` value accordingly before running migrations.

SECTION 10

Testing and Verification

Test Case	URL / Command	Expected Outcome	Result
Application build	<code>dotnet build</code>	Build succeeded, 0 errors	PASS
Database migration	<code>dotnet ef database update</code>	Both databases created with all tables	PASS
Home page	/	Landing page renders with navigation	PASS
Login — valid credentials	/Login/Index	Redirects to /Dashboard/Index	PASS
Login — invalid credentials	/Login/Index	Displays "Login Failed" message on same page	PASS
Books listing	/Books/Index	Paginated table renders (empty on fresh database)	PASS
Student listing	/Student/Index	Paginated student table renders	PASS
Librarian CRUD	/Librarian/Index	Add, edit, and delete operations function correctly	PASS
Dashboard statistics	/Dashboard/Index	All five counters display with zero on fresh database	PASS
Publications filter	/Publications/Index?type=Newspaper	Filtered publication list renders	PASS
Borrow workflow	/Borrow/Create?bookId=N	Form renders; submission marks book unavailable	PASS
Return workflow	/Borrow/Return?borrowRecordId=N	Confirmation page; submission restores availability	PASS

SECTION 11

Conclusion and Future Scope

11.1 Conclusion

The Library Management System capstone project demonstrates the successful application of full-stack web development concepts using the Microsoft .NET ecosystem. The project covers all core aspects of modern enterprise web application development:

- **Architectural discipline:** Strict adherence to the MVC pattern with clear layer separation through Controllers, Models, ViewModels, and Views.
- **Dual data access:** Effective use of both Entity Framework Core (ORM-based) and raw parameterised SQL, demonstrating proficiency with both abstracted and explicit data access approaches.
- **Security foundations:** AntiForgery token protection on all POST forms, parameterised queries preventing SQL injection, and ASP.NET Core Identity middleware integration.
- **Responsive design:** Bootstrap 5.3 integration ensuring a consistent, mobile-compatible user experience across all pages.
- **Database management:** Demonstrated competency with EF Core migrations for schema management across two separate SQL Server databases.
- **Software engineering practices:** Version control with Git, structured project layout, clean commit history, and published source code on GitHub.

All twelve modules have been implemented, tested, and verified against a live SQL Server Express instance. The project was completed without errors and the full source code is available at the repository listed on the cover page.

11.2 Future Scope

Enhancement	Description	Priority
Role-based Authorization	Implement Admin, Librarian, and Student roles using ASP.NET Core Identity with <code>[Authorize(Roles="...")]</code> attributes restricting access per role.	HIGH
Fine Management System	Automatically calculate overdue fines based on the difference between BorrowDate and ReturnDate, with configurable daily rates.	HIGH

Enhancement	Description	Priority
Email Notification Service	Integrate SMTP-based email notifications for due date reminders using ASP.NET Core's IEmailSender interface.	MEDIUM
Book Cover Image Upload	Allow administrators to upload book cover images stored in wwwroot/uploads with file size and type validation.	MEDIUM
REST API Layer	Expose a JSON REST API using ASP.NET Core Web API controllers to support potential mobile application development.	MEDIUM
Report Generation	Export borrow history, inventory, and overdue reports to Excel format using a library such as EPPlus or ClosedXML.	LOW
QR Code Integration	Generate unique QR codes for each book to facilitate rapid scanning at the library counter.	LOW

Table 11.1: Proposed future enhancements

Submitted by:

Repository:

Kavyansh Vats

github.com/Kavyansh180/

Registration No: 23BCE10301

Library-Management-System

Application No: IN26013045

B.Tech – Computer Science and Engineering

Page 10